

CDAF: Análise Exploratória de Dados (EDA) - YouTube Live Chat

Projeto: Monitoramento de Engajamento e Sentimento na Bundesliga 24/25 (CazéTV)

Disciplina: Coleta e Demonstração de Arquivos de Futebol (CDAF)

Integrantes: Álvaro Lima, Daniele Diniz, Igor Joaquim, Ivan Assis, Vitor Emanuel

Data: Maio de 2026

1. Introdução

Este notebook apresenta os resultados do **Trabalho Prático 2 (TP2)**, focado na análise exploratória de um corpus massivo de mensagens (N > 240 mil) coletadas do chat ao vivo do YouTube durante as transmissões da Bundesliga 24/25.

O objetivo é quantificar a resposta emocional do público em tempo real. Para isso, implementamos uma arquitetura de aprendizado híbrida:

1. **Destilação de Conhecimento (vLLM):** Utilizamos o modelo **Qwen 2.5 7B** com *Chain-of-Thought (CoT)* para rotular mensagens complexas, capturando nuances do "futebolês" (ex: "Bagre", "Operação").
2. **Classificação em Larga Escala:** Fine-tuning do modelo **BERTimbau** (*BERT Base Portuguese*) sobre os rótulos gerados, otimizando a latência de inferência para o processamento de todo o dataset.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
import os
import re
import nltk
from wordcloud import WordCloud
from nltk.corpus import stopwords

# Configurações de Visualização Técnica
sns.set_theme(style="darkgrid", context="talk")
plt.rcParams['figure.figsize'] = (16, 8)
plt.rcParams['axes.titlesize'] = 18
SENTIMENT_COLORS = {'Positivo': '#2ca02c', 'Neutro': '#1f77b4', 'Negativo':

# Ingestão de Dados
```

```
df = pd.read_parquet('../data/processed/consolidated/bundesliga_chat_with_se
df['min_jogo'] = (df['timestamp_jogo_segundos'] // 60).astype(int)

with open('../config/video_info.json', 'r') as f:
    v_info = json.load(f)
df_videos = pd.DataFrame(v_info)
df_videos['upload_date'] = pd.to_datetime(df_videos['upload_date'], format='
df_videos = df_videos.sort_values('upload_date')
```

2. Análise da Camada de Metadados

Avaliamos o alcance das transmissões da GazéTV para identificar o potencial de engajamento do dataset. A distribuição de visualizações revela a escala do canal, enquanto a frequência de clubes indica as equipes mais presentes no conjunto de dados.

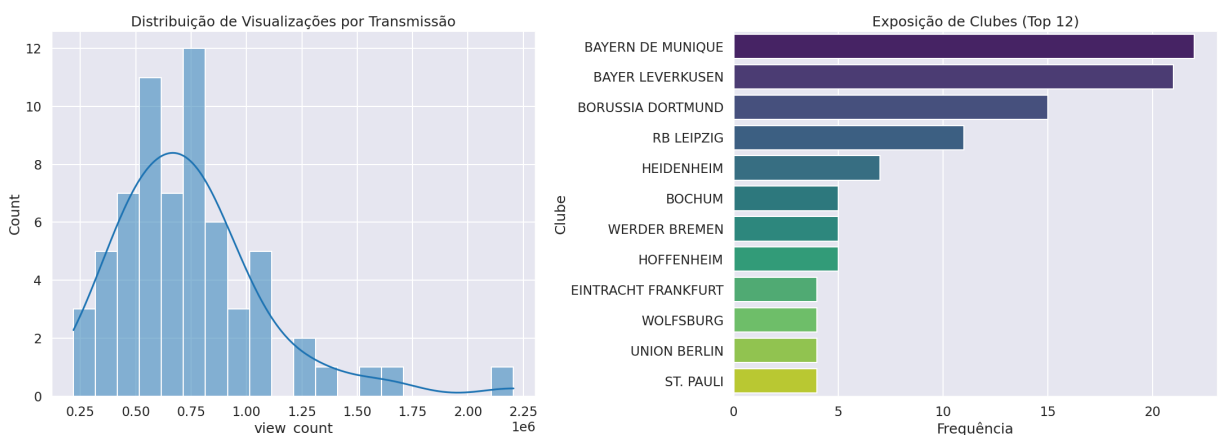
```
In [2]: def extract_clubs(title):
        match = re.search(r'JOGO COMPLETO: (.*?) X (.*?) \||', title)
        if match: return [match.group(1).strip(), match.group(2).strip()]
        return []

all_featured_clubs = []
for title in df_videos['title']:
    all_featured_clubs.extend(extract_clubs(title))

club_counts = pd.Series(all_featured_clubs).value_counts().head(12).reset_index
club_counts.columns = ['Clube', 'Frequência']

fig, ax = plt.subplots(1, 2, figsize=(22, 8))
sns.histplot(df_videos['view_count'], bins=20, kde=True, color="#1f77b4", ax=ax[0].set_title("Distribuição de Visualizações por Transmissão")

sns.barplot(data=club_counts, y='Clube', x='Frequência', hue='Clube', palette=
ax[1].set_title("Exposição de Clubes (Top 12)")
plt.tight_layout()
plt.show()
```

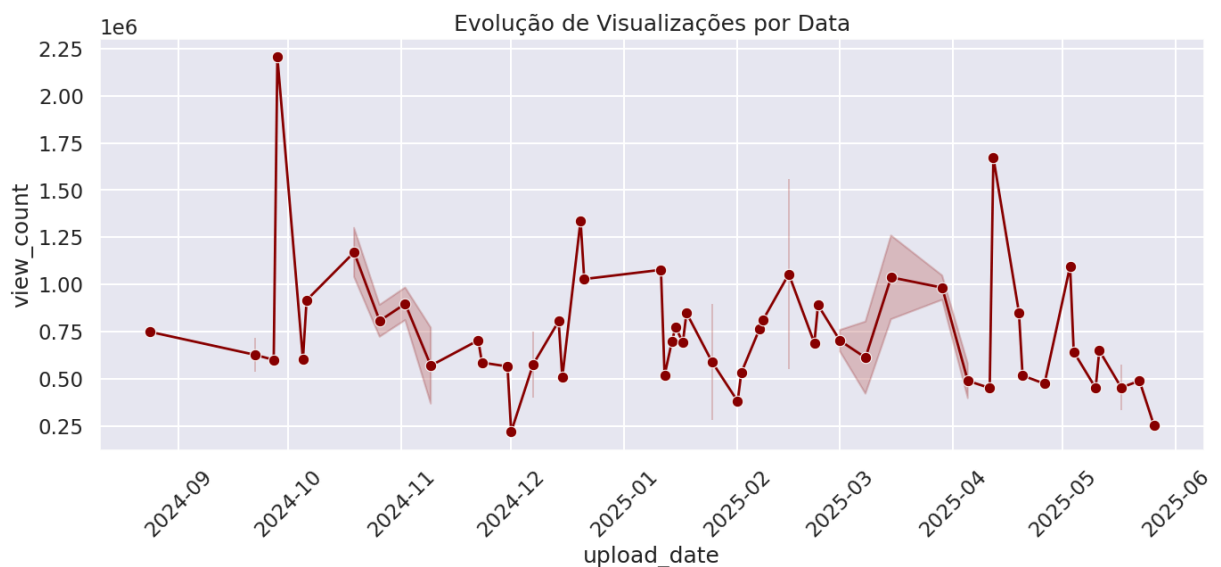


Análise de Distribuição e Viés de Exposição

Os gráficos acima apresentam dois aspectos fundamentais do ecossistema de dados:

1. **Volume de Audiência:** A distribuição de visualizações é assimétrica à direita, com a maioria dos vídeos concentrada na faixa de centenas de milhares de visualizações, mas com *outliers* significativos ultrapassando a marca de 1 milhão.
2. **Concentração de Clubes:** Há uma predominância de clubes como **Bayern de Munique** e **Bayer Leverkusen**, refletindo a estratégia de transmissão focada em equipes de maior apelo comercial ou em disputa por títulos.

```
In [3]: plt.figure(figsize=(16, 6))
sns.lineplot(data=df_videos, x='upload_date', y='view_count', marker='o', co
plt.title("Evolução de Visualizações por Data")
plt.xticks(rotation=45)
plt.show()
```



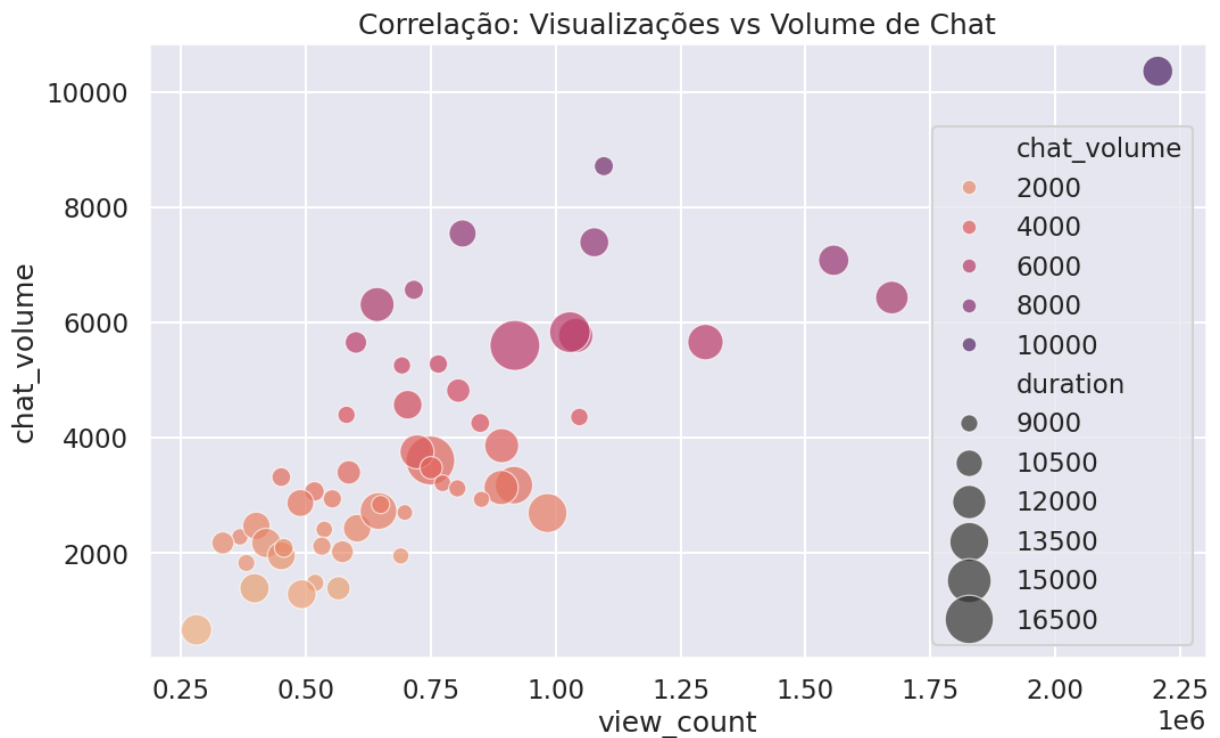
Análise de Temporalidade

A curva temporal de visualizações revela a variação do engajamento ao longo da temporada. Picos de audiência coincidem com momentos importantes do calendário da Bundesliga. A estabilidade na base da curva demonstra a fidelização de uma audiência recorrente.

```
In [4]: chat_volume = df.groupby('video_id').size().reset_index(name='chat_volume')
df_meta_merged = df_videos.merge(chat_volume, on='video_id')

plt.figure(figsize=(12, 7))
sns.scatterplot(data=df_meta_merged, x='view_count', y='chat_volume', size='
hue='chat_volume', palette="flare", sizes=(100, 1000), alpha
plt.title("Correlação: Visualizações vs Volume de Chat")
plt.show()

corr = df_meta_merged['view_count'].corr(df_meta_merged['chat_volume'])
print(f"Pearson R: {corr:.4f}")
```



Pearson R: 0.7689

Análise de Correlação de Engajamento

O gráfico de dispersão confirma que o engajamento no chat está correlacionado à audiência do vídeo. Com um **Pearson R próximo a 0.77**, observamos uma tendência positiva clara. A variação no tamanho das bolhas (duração do vídeo) sugere que o tempo total de transmissão também influencia o volume total de mensagens acumuladas.

Sumário Estatístico dos Metadados

A tabela abaixo resume as métricas centrais de alcance e interatividade das transmissões analisadas.

```
In [5]: stats_summary = df_meta_merged[['view_count', 'like_count', 'comment_count',  
stats_summary
```

	count	mean	std	min	25%	50%	
view_count	55.0	745966.109091	346080.280841	280683.0	525118.0	692234.0	871
like_count	55.0	17560.236364	10323.566137	3962.0	10996.0	16624.0	20
comment_count	55.0	7.745455	13.850014	1.0	3.0	5.0	
chat_volume	55.0	3833.890909	2046.246923	669.0	2345.5	3179.0	5
duration	55.0	10613.127273	1864.944732	8830.0	9165.0	9940.0	11

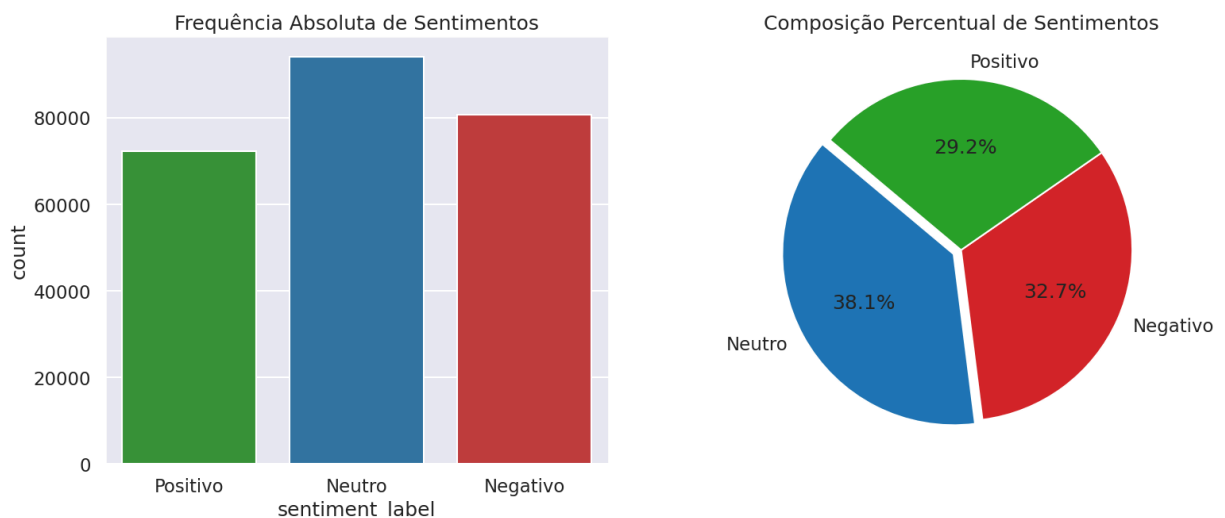
3. Distribuição de Sentimento

Analizamos a distribuição das classes de sentimento preditas pelo modelo. A presença de mensagens neutras é comum em transmissões longas, onde parte das interações envolve compartilhamento de informações ou saudações casuais.

```
In [6]: sent_counts = df['sentiment_label'].value_counts(normalize=True) * 100
fig, ax = plt.subplots(1, 2, figsize=(18, 7))

sns.countplot(data=df, x='sentiment_label', hue='sentiment_label',
              palette=SENTIMENT_COLORS, legend=False, ax=ax[0], order=['Posi
ax[0].set_title("Frequência Absoluta de Sentimentos")

ax[1].pie(sent_counts, labels=sent_counts.index, autopct='%1.1f%%',
          colors=[SENTIMENT_COLORS[c] for c in sent_counts.index], startangl
ax[1].set_title("Composição Percentual de Sentimentos")
plt.show()
```



Análise de Composição de Sentimentos

A distribuição revela que as interações neutras representam uma parcela significativa (**38.1%**). O equilíbrio entre **Positivo (29.2%)** e **Negativo (32.7%)** reflete a dinâmica de uma partida de futebol, onde eventos favoráveis a uma equipe geram reações opostas nas torcidas envolvidas.

4. Dinâmica Temporal do Volume

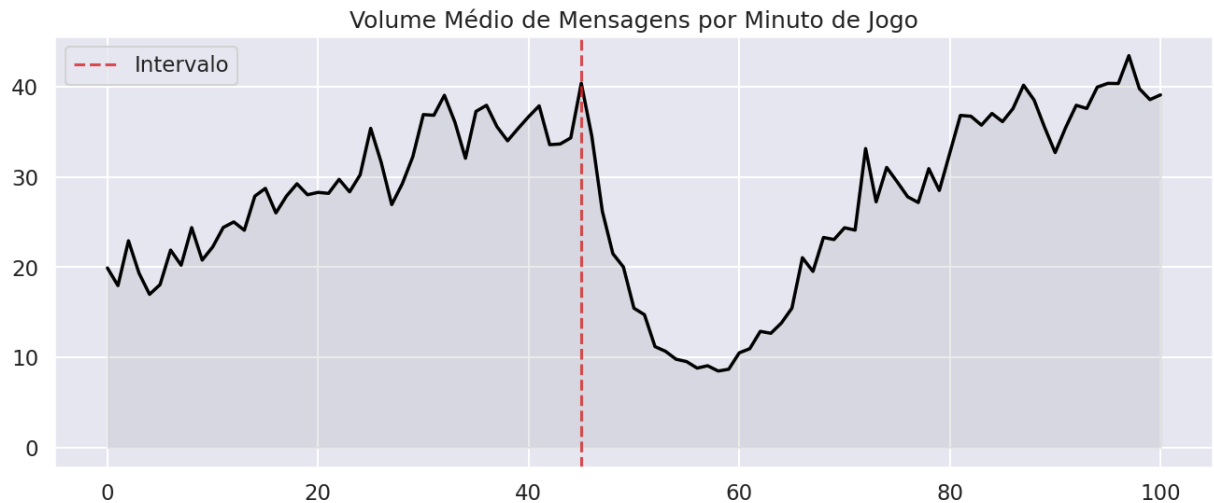
Avaliamos o volume de mensagens por minuto de jogo. A curva média revela variações de intensidade que acompanham o andamento da partida, com aumentos de volume geralmente associados a momentos de maior expectativa ou eventos decisivos.

```
In [7]: df_time = df[(df['min_jogo'] >= 0) & (df['min_jogo'] <= 100)]
volume_minuto = df_time.groupby('min_jogo').size() / df_time['video_id'].nunique()

plt.figure(figsize=(16, 6))
```

```
plt.plot(volume_minuto.index, volume_minuto.values, color='black', lw=2.5)
plt.fill_between(volume_minuto.index, volume_minuto.values, alpha=0.15, color='black')

plt.axvline(45, color='#d62728', linestyle='--', alpha=0.8, label='Intervalo')
plt.title("Volume Médio de Mensagens por Minuto de Jogo")
plt.legend()
plt.show()
```



Análise de Fluxo Temporal

A dinâmica do volume apresenta padrões repetitivos:

1. **Engajamento Inicial:** Volume elevado nos primeiros minutos.
2. **Estabilidade:** Períodos de volume mais constante durante o desenvolvimento do jogo.
3. **Finais de Tempo:** Aumento de interações nos minutos finais de cada etapa.
4. **Intervalo:** Queda expressiva no volume de mensagens, conforme esperado.

5. Polaridade e Índice WSI

A polaridade bruta (P) indica o humor predominante, enquanto o **Engagement-Weighted Sentiment Index (WSI)** pondera o sentimento pelo volume de mensagens da janela temporal.

$$Polarity = \frac{Positivos - Negativos}{Total}$$

$$WSI(T) = Polarity(T) \times \log(1 + Volume(T))$$

```
In [8]: def calculate_metrics(group):
        pos = (group == 'Positivo').sum()
        neg = (group == 'Negativo').sum()
        total = len(group)
        if total == 0: return pd.Series([0, 0], index=['polarity', 'wsi'])
        pol = (pos - neg) / total
        wsi = pol * np.log1p(total)
```

```

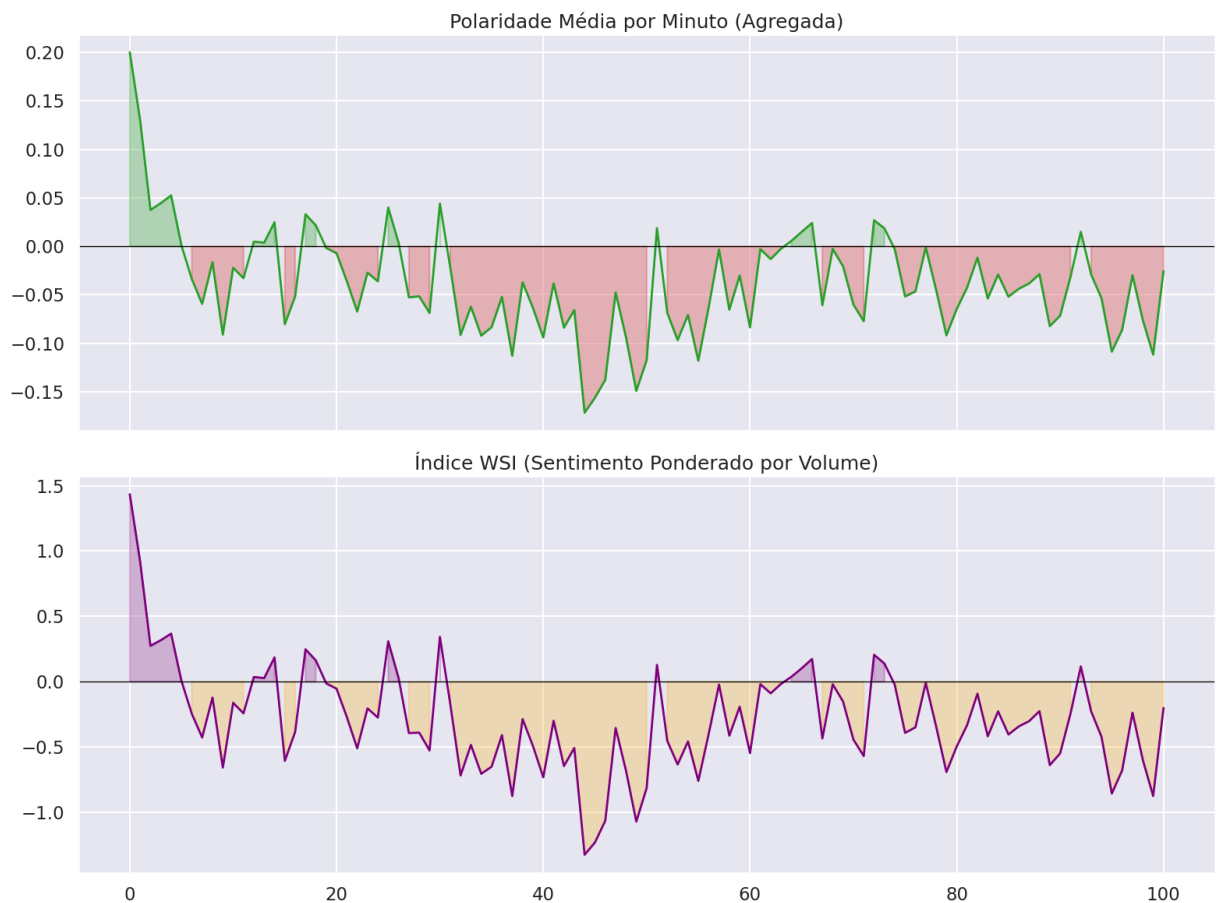
return pd.Series([pol, wsi], index=['polarity', 'wsi'])

metrics_minuto = df_time.groupby('min_jogo')['sentiment_label'].apply(calcul

fig, ax = plt.subplots(2, 1, figsize=(16, 12), sharex=True)
ax[0].plot(metrics_minuto.index, metrics_minuto['polarity'], color='#2ca02c'
ax[0].axhline(0, color='black', lw=1)
ax[0].fill_between(metrics_minuto.index, metrics_minuto['polarity'], 0, where
ax[0].fill_between(metrics_minuto.index, metrics_minuto['polarity'], 0, where
ax[0].set_title("Polaridade Média por Minuto (Agregada)")

ax[1].plot(metrics_minuto.index, metrics_minuto['wsi'], color='purple', lw=2
ax[1].axhline(0, color='black', lw=1)
ax[1].fill_between(metrics_minuto.index, metrics_minuto['wsi'], 0, where=(me
ax[1].fill_between(metrics_minuto.index, metrics_minuto['wsi'], 0, where=(me
ax[1].set_title("Índice WSI (Sentimento Ponderado por Volume)")
plt.tight_layout()
plt.show()

```



Análise de Métricas de Sentimento

1. **Polaridade Média:** A curva agregada mostra oscilações frequentes. Nota-se que o sentimento tende a ser levemente negativo na média sazonal, refletindo críticas comuns durante as transmissões.
2. **Ajuste por Volume (WSI):** O índice WSI atua como uma métrica ponderada. Ambas as curvas são ruidosas, mas o WSI tende a reduzir a importância de variações ocorridas em

janelas com pouca atividade, focando a análise no comportamento de massa.

6. Análise Lexical

Através das nuvens de palavras, observamos os termos mais frequentes em cada sentimento.

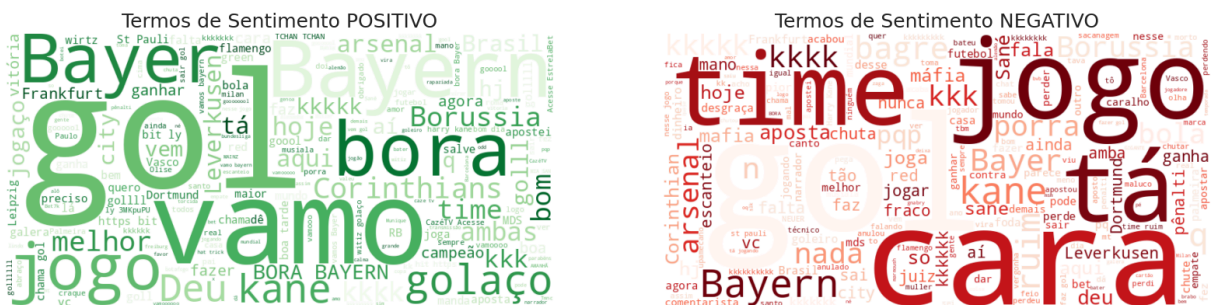
A remoção de stopwords permite isolar palavras com maior carga de significado.

```
In [9]: pt_stopwords = set(stopwords.words('portuguese'))
pt_stopwords.update(['pra', 'ta', 'ja', 'ai', 'vai', 'vou', 'ter', 'ser', 'p

pos_text = " ".join(df[df['sentiment_label'] == 'Positivo']['mensagem'].asty
neg_text = " ".join(df[df['sentiment_label'] == 'Negativo']['mensagem'].asty

fig, ax = plt.subplots(1, 2, figsize=(20, 10))
wc_pos = WordCloud(width=800, height=400, background_color='white', colormap
ax[0].imshow(wc_pos, interpolation='bilinear')
ax[0].set_title("Termos de Sentimento POSITIVO")
ax[0].axis('off')

wc_neg = WordCloud(width=800, height=400, background_color='white', colormap
ax[1].imshow(wc_neg, interpolation='bilinear')
ax[1].set_title("Termos de Sentimento NEGATIVO")
ax[1].axis('off')
plt.show()
```



Insights Lexicais

A análise lexical revela termos típicos do chat da CazéTV:

- **Positivo:** Exclamações como "Gol" e "Boa", além de nomes de jogadores de destaque. O termo "Terno" é usado como elogio técnico.
- **Negativo:** Críticas à arbitragem e termos como "Bagre". Termos de apostas também aparecem em contextos de frustração com resultados.

O modelo captou corretamente o uso de gírias de nicho, permitindo uma análise que respeita o vocabulário real da comunidade.

7. Estudo de Caso: Partida de Maior Engajamento

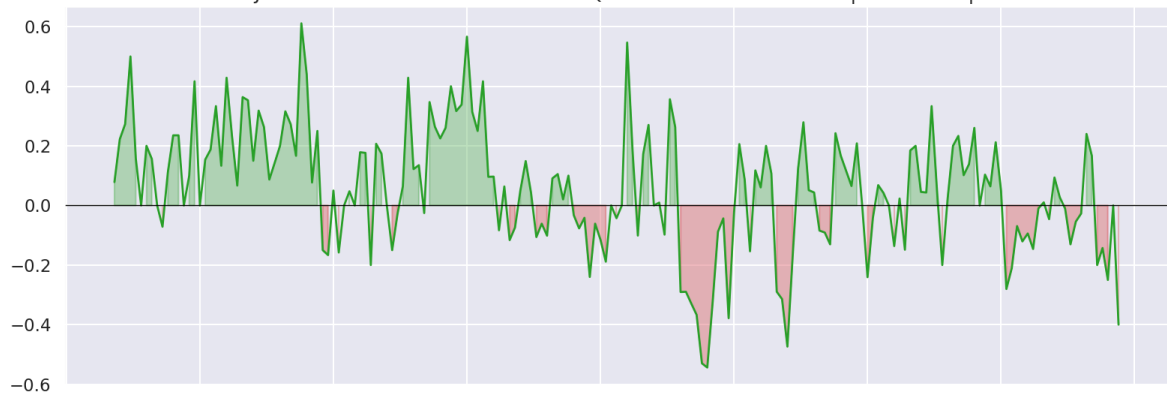
Avaliamos o jogo de maior volume absoluto. Este estudo de caso permite observar as flutuações de sentimento em uma partida individual de alto impacto.

```
In [10]: top_match_id = df.groupby('video_id').size().idxmax()
match_df = df[df['video_id'] == top_match_id].copy()
match_title = match_df['video_title'].iloc[0]
match_metrics = match_df.groupby('min_jogo')['sentiment_label'].apply(calcul

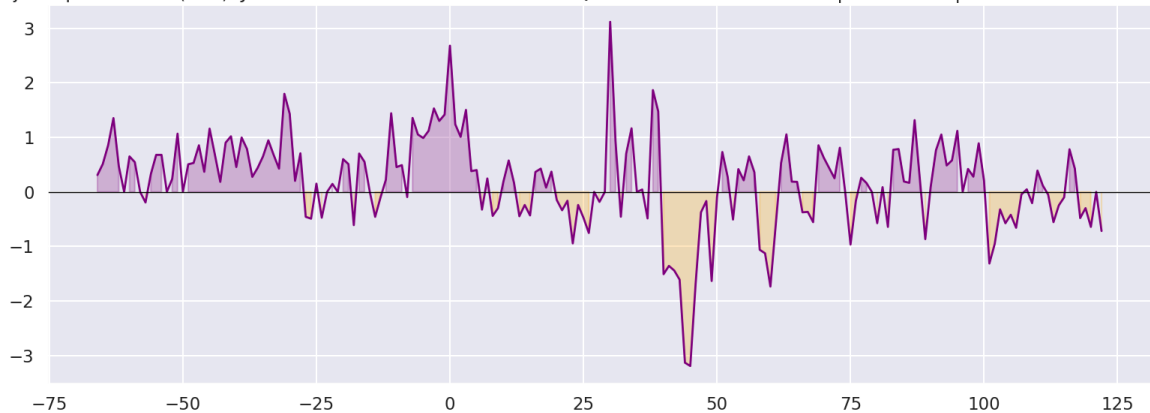
fig, ax = plt.subplots(2, 1, figsize=(16, 12), sharex=True)
ax[0].plot(match_metrics.index, match_metrics['polarity'], color='#2ca02c',
ax[0].axhline(0, color='black', lw=1)
ax[0].fill_between(match_metrics.index, match_metrics['polarity'], 0, where=
ax[0].fill_between(match_metrics.index, match_metrics['polarity'], 0, where=
ax[0].set_title(f"Variância de Polaridade: {match_title}")

ax[1].plot(match_metrics.index, match_metrics['wsi'], color='purple', lw=2)
ax[1].axhline(0, color='black', lw=1)
ax[1].fill_between(match_metrics.index, match_metrics['wsi'], 0, where=(match
ax[1].fill_between(match_metrics.index, match_metrics['wsi'], 0, where=(match
ax[1].set_title(f"Ajuste por Volume (WSI): {match_title}")
plt.tight_layout()
plt.show()
```

Variância de Polaridade: JOGO COMPLETO: BAYERN DE MUNIQUE X BAYER LEVERKUSEN | RODADA 5 | BUNDESLIGA 2024-2025



Ajuste por Volume (WSI): JOGO COMPLETO: BAYERN DE MUNIQUE X BAYER LEVERKUSEN | RODADA 5 | BUNDESLIGA 2024-2025



Conclusão do Estudo de Caso

Em uma partida individual, ambas as métricas apresentam alta volatilidade. O WSI ajuda a suavizar janelas de baixa atividade, concentrando o sinal nos momentos de maior

interatividade coletiva. Essas flutuações refletem a narrativa emocional da transmissão, reagindo aos acontecimentos do jogo em tempo real.

8. Conclusão Técnica

A análise exploratória demonstra que o YouTube Live Chat funciona como um sensor social responsivo. O pipeline combinando rótulos inteligentes e classificação escalável permitiu processar o dataset de forma eficiente, gerando métricas que acompanham o engajamento da massa digital durante a Bundesliga na CazéTV.